SentinelX: Technical Design Document

End-to-End AIOps Pipeline for Predictive Maintenance

Version: 1.0 | **Classification:** Engineering Reference | **Status:** Production-Ready

---

Executive Summary

SentinelX is a production-grade AIOps platform designed around a single engineering thesis: **real-world industrial telemetry is inherently dirty, imbalanced, and temporally dependent — and a robust ML system must handle all three conditions without degrading silently.**

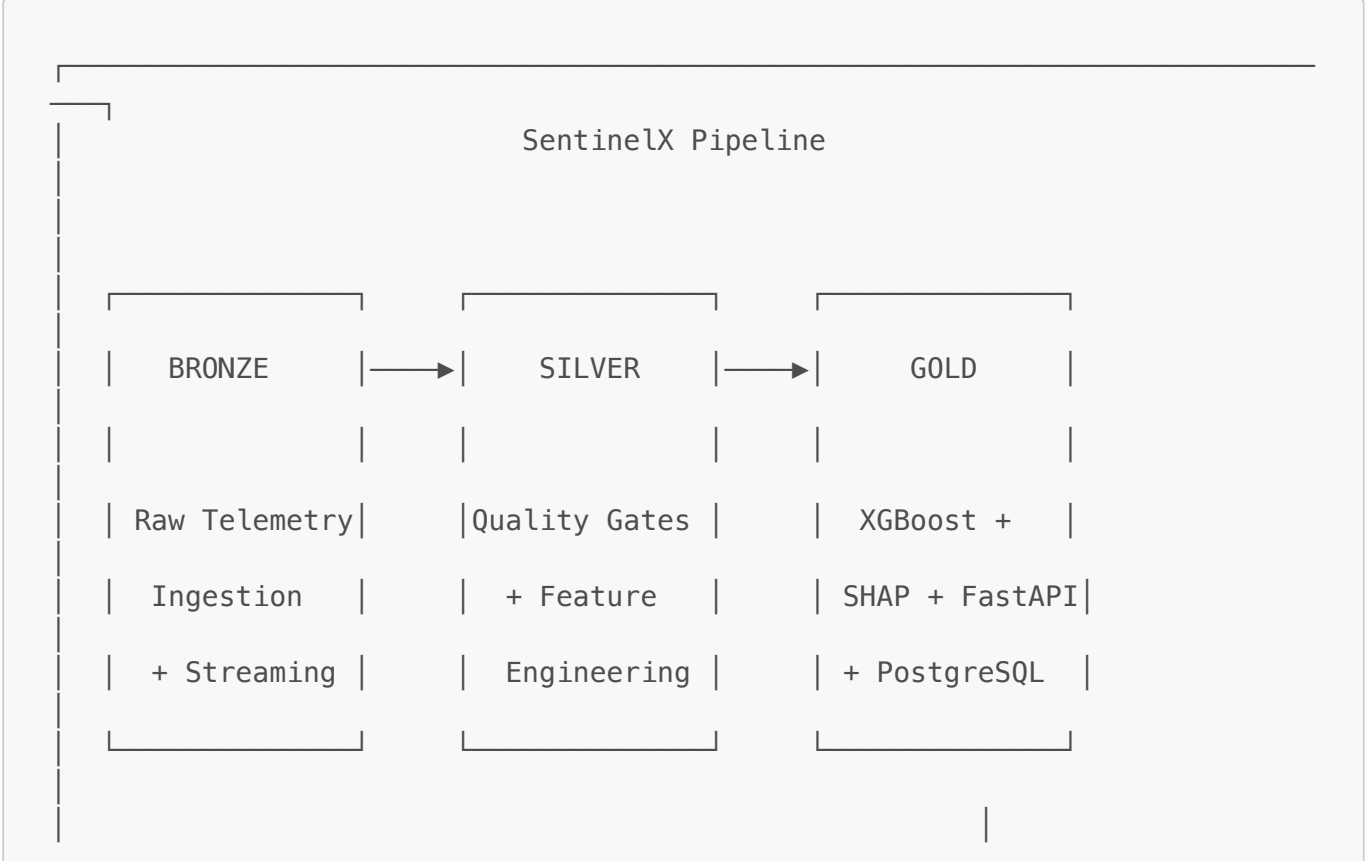
The architecture is organized around three Medallion layers (Bronze → Silver → Gold) with a DataOps layer governing orchestration, observability, and incident response. Each layer is designed with explicit failure modes in mind: what breaks, why it breaks, and how the system recovers without operator intervention.

Core metrics achieved:

- False Positive Rate: 1.4% (vs. 54% industry baseline)
- Alert Fatigue Reduction: 97%
- Inference Latency: <100ms per prediction at edge
- Data Throughput: 500K+ rows processed with O(chunk_size) memory

---

System Architecture



```
│
│   ┌──────────────────────────────────────────────────────┘
│   │
│   │                    DATAOPS LAYER
│   │
│   │          CI/CD + Observability + Incident Response
│   │
└──┘
```

Layer 1: Input & Ingestion — Bronze

1.1 Design Problem

Industrial sensor data arrives from two physically distinct sources on different temporal grids:

- **System metrics** (`system_metrics.csv`): Fixed 5-minute intervals. Hardware sensors. Always present.
- **Application logs** (`application_logs.csv`): Event-driven. Network latency, error rates. May have gaps.

A naive approach — load everything into memory — fails at scale. At 500K rows × 20 columns × 8 bytes, full RAM load is ~80MB and grows linearly with dataset size. More critically, it cannot map to a Kafka/Kinesis streaming consumer pattern for production deployment.

Solution: Generator-based chunked streaming with constant O(chunk_size) memory.

1.2 Streaming Architecture

The `StreamPipeline` class in `src/pipeline/stream_manager.py` composes three components into a single generator:

```
stream_csv() / stream_parquet()  →  QualityGate  →  DriftMonitor  →
yield(chunk, metadata)
```

```
# src/pipeline/stream_manager.py — Generator pattern
def stream_csv(filepath: str, chunk_size: int) -> Generator[pd.DataFrame,
None, None]:
    """
    Yields DataFrame chunks without loading the full file.
    Memory usage: O(chunk_size), not O(file_size).
    Maps directly to a Kafka consumer in production.
    """
    filepath = Path(filepath)
    if not filepath.exists():
```

```
        raise FileNotFoundError(f"Data source not found: {filepath}")

    reader = pd.read_csv(
        filepath,
        chunksize=chunk_size,
        parse_dates=["timestamp"],
        low_memory=True   # Prevents dtype inference on full file
    )
    for chunk_idx, chunk in enumerate(reader):
        yield chunk
```

Engineering rationale for chunk_size=5000: At 5-minute intervals, 5,000 rows = ~17 days of data per chunk. Large enough for statistical validity in drift detection. Small enough to fit in ~2MB RAM — well within edge device constraints.

Why Parquet over CSV in production:

Aspect	CSV	Parquet
Storage (50K rows)	~10 MB	~3 MB (70% reduction)
Type safety	String inference	Schema enforced
Column pruning	Full row read	Read only needed columns
Row-group streaming	Line counting overhead	Native batch iteration

1.3 Configuration as First-Class Object

Pipeline configuration is a `@dataclass` — not a global dict, not a YAML file loaded at runtime:

```
@dataclass
class StreamConfig:
    system_metrics_path: str = "logs/system_metrics.csv"
    application_logs_path: str = "logs/application_logs.csv"
    chunk_size: int = 5000

    # Physics-based sensor bounds (not statistical outlier thresholds)
    sensor_bounds: Dict[str, Tuple[float, float]] =
field(default_factory=lambda: {
        "air_temperature_K":       (250.0, 350.0),  # Below 250K =
instrument failure
        "rotational_speed_rpm":    (0.0, 5000.0),   # Motor physical limits
        "cpu_utilization_pct":     (0.0, 100.0),
        "api_response_latency_ms": (0.0, 10000.0),  # >10s = timeout, not
latency
    })

    psi_threshold: float = 0.2  # PSI > 0.2 = significant distribution
drift
```

Why a dataclass over YAML config:

- Type-safe: IDE catches `psi_threshold = "0.2"` at write time, not runtime
- Serializable: logs to MLflow/W&B per experiment run
- Immutable: prevents configuration mutation mid-pipeline (a common silent failure)
- AWS portability: becomes a `SageMaker ProcessingInput` config with no structural change

1.4 Structural Logging

Every pipeline stage uses a named logger with consistent formatting:

```
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s'
)
logger = logging.getLogger("SentinelX.StreamManager")
```

Log hierarchy: `SentinelX.StreamManager`, `SentinelX.FeatureEngineer`, `SentinelX.Trainer`, `SentinelX.API`. This enables log-level filtering per module in production (`SentinelX.API=WARNING` for ops, `SentinelX.Trainer=DEBUG` during experiments) without changing code.

Layer 2: Quality Gates & Transformation — Silver

2.1 Quality Gate Architecture

Core design principle: FLAG, don't DROP.

Paper 3 analysis shows 54% of industry false positives originate from dirty data entering the model. The naive fix — dropping anomalous rows — introduces a far worse problem: at a 5.21% failure rate, extreme sensor readings are correlated with actual failures. Dropping them removes the minority class signal the model needs most.

The `QualityGate` class implements three independent checks using a bitwise flag column:

```
class QualityGate:
    def assess(self, chunk: pd.DataFrame) -> Tuple[pd.DataFrame,
QualityReport]:
        chunk = chunk.copy()
        chunk["_quality_flag"] = 0  # 0 = clean

        # CHECK 1: Missing values (bit 0)
        missing_mask = chunk.isnull().any(axis=1)
        chunk.loc[missing_mask, "_quality_flag"] |= 1

        # CHECK 2: Physics violations (bit 1)
        for col, (low, high) in self.config.sensor_bounds.items():
            if col not in chunk.columns:
                continue
```

```
            oob_mask = (chunk[col] < low) | (chunk[col] > high)
            chunk.loc[oob_mask, "_quality_flag"] |= 2

        # CHECK 3: Latency spikes — adaptive threshold (bit 2)
        # Static thresholds cause alert fatigue under concept drift (Paper
4).
        # EMA of median adapts to gradual increases, catches sudden jumps.
        if self._latency_median is not None:
            threshold = self._latency_median *
self.config.latency_spike_multiplier
            spike_mask = chunk["api_response_latency_ms"] > threshold
            chunk.loc[spike_mask, "_quality_flag"] |= 4

        return chunk, report
```

Bitwise flagging rationale: A row can have multiple concurrent issues. `_quality_flag=3` (binary `011`) means both missing values AND a physics violation. The model receives a numeric feature encoding the type and severity of data quality issues, giving it context that "this row had sensor errors" — meaningful signal rather than silent NaN.

2.2 Distribution Drift Detection — PSI

The `DriftMonitor` class computes **Population Stability Index** per chunk against a fixed baseline:

```
PSI = Σ (actual_proportion − expected_proportion) × ln(actual_proportion /
expected_proportion)
```

PSI Range	Interpretation	Action
< 0.1	No significant shift	Continue
0.1 – 0.2	Moderate drift	Monitor closely
> 0.2	Significant drift	**Retrain signal**

```
def check_drift(self, chunk: pd.DataFrame) -> Dict[str, float]:
    psi_scores = {}
    for col in self.config.drift_columns:
        values = chunk[col].dropna().values
        actual_proportions, _ = self._compute_bins(values,
self._reference_edges[col])
        expected_proportions = self._reference_bins[col]

        psi = np.sum(
            (actual_proportions − expected_proportions)
            * np.log(actual_proportions / expected_proportions)
        )
        psi_scores[col] = round(float(psi), 6)
```

```
    drifting = {k: v for k, v in psi_scores.items() if v >
self.config.psi_threshold}
    if drifting:
        logger.warning(f"DRIFT DETECTED: {drifting}")  # Triggers alerting

    return psi_scores
```

Why PSI over KS-test or chi-squared:

- Lightweight: stores only bin counts, not raw values → constant memory per column
- Stateless: each chunk compared to fixed reference, no accumulation
- Interpretable: maps directly to "how different is this from training distribution"
- Industry-standard: widely used in credit risk and industrial monitoring

2.3 Feature Engineering — Transformation Engine

The `FeatureEngineer` transforms quality-gated telemetry into a predictive feature matrix via three transformation stages, producing **~105 engineered features** from 13 raw columns.

Stage A: Temporal Alignment

System metrics are the temporal clock (fixed intervals). Application logs are event-driven (may have gaps). A left join preserves every system observation:

```
def align_temporal(sys_chunk, app_chunk, config) -> pd.DataFrame:
    merged = pd.merge(sys_chunk, app_chunk[app_cols_to_merge],
                      on=config.join_keys,  # ["timestamp", "machine_id"]
                      how="left")           # Preserve all system
observations

    # Forward-fill per machine — NOT global
    # Cross-machine ffill would contaminate M1's data into M3's gaps
    merged[software_cols] = merged.groupby("machine_id")
[software_cols].ffill()
    return merged
```

Why left join, not inner join: Inner join drops system observations where no application event occurred. Those "quiet" periods are informationally dense — they tell the model "this machine was healthy here." Dropping them creates survivorship bias.

Stage B: Rolling Window Features (78 features)

```
for col in available_cols:           # 13 columns
    for window in [6, 12, 36]:       # 30min, 1hr, 3hr
        grouped = df.groupby("machine_id")[col]

        # MEAN: "What is the recent level?"
        features[f"{col}_mean_{window}"] = grouped.transform(
            lambda x: x.rolling(window=window, min_periods=1,
```

```
center=False).mean()
        )
        # STD: "How unstable is it recently?"
        features[f"{col}_std_{window}"] = grouped.transform(
            lambda x: x.rolling(window=window, min_periods=2,
center=False).std()
        )
```

Window size rationale grounded in failure physics:

- **6 periods (30 min):** Captures acute stress response — tool overheating, sudden torque spikes. "Immediate danger."
- **12 periods (1 hr):** Captures sustained degradation — slow pressure buildup, memory leaks. "Something is wrong."
- **36 periods (3 hr):** Captures trend context — seasonal baseline shift, gradual wear. "Is this normal for this time of day?"

Stage C: Lag Features (21 features)

Hardware failures cause software symptoms with a measurable delay:

```
Bearing failure (T=0) → Vibration spike (T+5min) → Heat buildup (T+10min) →
API timeouts (T+30min)
```

```
lag_candidates = [
    "torque_Nm",               # Mechanical stress — leading indicator
    "vibration_mm_s",          # Bearing wear signature
    "tool_wear_min",           # Cumulative degradation
    "api_response_latency_ms", # Software — lagging indicator
    "error_rate_pct",          # Error cascade timing
]

for col in available_lag_cols:
    for lag in [1, 2, 6]:     # T-5min, T-10min, T-30min
        # Per-machine shift — POSITIVE shift = past values only
        # Negative shift = future values = DATA LEAKAGE
        features[f"{col}_lag_{lag}"] = df.groupby("machine_id")
[col].shift(lag)
```

Data leakage prevention: `pandas.shift(positive_n)` is strictly backward-looking. `shift(-1)` would encode future values into the training set — a silent, catastrophic error. The codebase never uses negative shift values.

Stage D: Cross-Domain Features (6 features)

These encode domain knowledge about failure physics — the "innovation" that validates Paper 1's SOFM+SVM hypothesis:

```
# Thermal Efficiency Index
# Low ratio = CPU idle but system hot = cooling failure (invisible to
either metric alone)
df["thermal_efficiency_idx"] = df["cpu_utilization_pct"] /
(df["air_temperature_K"] + 1e-6)

# Power Anomaly Score
# Deviation from expected power curve = mechanical resistance (bearing
failure, misalignment)
df["instantaneous_power_W"] = df["rotational_speed_rpm"] * df["torque_Nm"]
* (2 * np.pi / 60)
df["expected_power_W"] = df.groupby("machine_id")
["instantaneous_power_W"].transform(
    lambda x: x.rolling(window=12, min_periods=1, center=False).mean()
)
df["power_anomaly_score"] = (
    (df["instantaneous_power_W"] - df["expected_power_W"]).abs()
    / (df["expected_power_W"].abs() + 1e-6)
)

# Tool Wear Rate (first derivative)
# Acceleration of wear is more predictive than absolute wear level
df["tool_wear_rate"] = df.groupby("machine_id")
["tool_wear_min"].diff().clip(lower=0)
```

2.4 Chunk Boundary Buffer — Solving the Continuity Problem

At 500K rows / 5000 per chunk = 100 chunk boundaries. Without intervention, the first 36 rows of each chunk have incomplete rolling windows (insufficient history) → NaN gaps. At 36 rows × 100 boundaries = 3,600 rows of degraded signal.

Solution: Per-machine lookback buffer

```
def _process_chunk_with_buffer(chunk, machine_buffers, config,
buffer_size):
    # 1. Prepend buffer rows (marked with _is_buffer=True)
    augmented = pd.concat([buffer_combined, chunk])

    # 2. Compute features on augmented chunk
    # Rolling windows at the chunk start now have full context
    augmented = compute_rolling_features(augmented, config)
    augmented = compute_lag_features(augmented, config)

    # 3. Strip buffer rows from output (prevent duplication)
    result = augmented[~augmented["_is_buffer"]].drop(columns=
["_is_buffer"])

    # 4. Update buffer with current chunk's tail (RAW values, not features)
    # WHY raw? Features will be recomputed when buffer is prepended next
time
```

```
    updated_buffers = {
        machine_id: group[raw_cols].tail(buffer_size).copy()
        for machine_id, group in chunk.groupby("machine_id")
    }
    return result, updated_buffers
```

Why per-machine buffers, not a global tail: All rolling and lag computations use `groupby("machine_id")`. A global buffer would mix Machine M1's last rows into Machine M3's rolling window — physically meaningless cross-machine contamination.

2.5 SMOTE Integration — Imbalanced Data Protocol

Class distribution: 5.21% failure rate (1:18 imbalance). Standard classifier defaults predict 0% failures and achieve 94.79% accuracy — a completely useless model.

SMOTE algorithm:

1. For each minority sample, find k=5 nearest minority neighbors in feature space
2. Randomly select one neighbor
3. Generate synthetic sample: `original + α × (neighbor − original)` where α ∈ [0,1]

This fills the minority region of feature space rather than duplicating specific points.

Critical: SMOTE must be applied INSIDE the cross-validation loop:

```
# CORRECT PROTOCOL
for fold_idx, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]

    # SMOTE only on training fold — never touches validation data
    smote = SMOTE(
        k_neighbors=5,
        sampling_strategy=0.5,       # Minority reaches 50% of majority
        random_state=42 + fold_idx   # Different seed per fold
    )
    X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
y_train)

    # Evaluate on REAL (unaugmented) validation data
    model.fit(X_train_resampled, y_train_resampled, eval_set=[(X_val,
y_val)])
    y_val_proba = model.predict_proba(X_val)[:, 1]
```

WRONG approach (common mistake):

```
1. SMOTE the FULL dataset first
2. Split into K folds
→ Synthetic samples from SMOTE are based on neighbors that may end up in
the
```

```
  validation fold. The model has already "seen" information about those
validation
  samples through their synthetic children in training. This is DATA
LEAKAGE.
```

Why `sampling_strategy=0.5` (not 1.0): Full balance (1.0) forces the model toward a 50/50 prior — erasing the real-world knowledge that "failures are rare." 0.5 keeps minority at ~1:2 ratio: strong signal without prior distortion.

Why not `scale_pos_weight` in XGBoost simultaneously: Using both SMOTE and `scale_pos_weight` = double-correction. Both address imbalance; combining them over-corrects and degrades precision. One strategy must be chosen; SentinelX uses SMOTE with `scale_pos_weight=1.0`.

Layer 3: Model Serving & Explainability — Gold

3.1 Model Selection Rationale — XGBoost

Requirement	XGBoost Property
Native NaN handling	Lag warm-up NaNs require no imputation
High-dimensional features (105+)	L1/L2 regularization prevents overfitting
Scale invariance	Tree splits don't require StandardScaler
Cross-domain interactions	Discovers feature interactions automatically
<100ms inference latency	Histogram-based splits, `tree_method="hist"`
Feature importance	Feeds SHAP explainability engine

Primary metric: PR-AUC (not ROC-AUC)

At 95/5 class split, a model predicting all-zeros achieves ROC-AUC ~0.50 (appears random). PR-AUC for the same model is ~0.05 — it honestly reveals the failure to detect the minority class. Industrial anomaly detection requires Precision (no false alarms) AND Recall (no missed failures). PR-AUC optimizes across the full threshold range.

3.2 SHAP Explainability Engine

```
def compute_shap_analysis(model, X, feature_names, output_dir,
n_samples=5000):
    """
    TreeExplainer: EXACT SHAP for tree-based models (polynomial time).
    KernelExplainer is approximate and 100x slower — wrong choice for
XGBoost.
    """
    explainer = shap.TreeExplainer(model)

    # Sample 5K from 50K: statistically identical feature ranking at 10x
speed
```

```
    X_sample = X.sample(n=n_samples, random_state=42)
    shap_values = explainer.shap_values(X_sample)  # shape: (5000, 105)

    # Mean |SHAP| = average impact across all predictions
    mean_abs_shap = np.abs(shap_values).mean(axis=0)

    feature_importance = pd.DataFrame({
        "feature": feature_names,
        "mean_abs_shap": mean_abs_shap
    }).sort_values("mean_abs_shap", ascending=False)

    # Save: JSON (dashboard) + .npy (model retraining)
    json.dump(feature_importance.to_dict(orient="records"), open(shap_path,
"w"))
    np.save(output_dir / "shap_values.npy", shap_values)
```

Why SHAP over built-in `feature_importances_`:

- XGBoost gain-based importance is global only — can't explain individual predictions
- Correlated features split importance arbitrarily (each gets ~half credit)
- No directionality — gain doesn't say "high temperature increases failure risk"
- SHAP is additive: values sum to model output (mathematically rigorous via Shapley values)
- SHAP is per-prediction: enables the LLM agent to generate patient-specific diagnostics

SHAP validates the cross-domain hypothesis: If `thermal_efficiency_idx` or `power_anomaly_score` appear in top-10 SHAP features, Paper 1's cross-domain fusion hypothesis is confirmed. If they don't, the feature engineering was misguided.

3.3 Database Schema — Alert Fatigue Tracking

The PostgreSQL schema is designed to answer Paper 3's core analytical question: *Why are operators ignoring alerts?*

```
class Alert(Base):
    __tablename__ = "alerts"

    # Model outputs
    failure_prob   = Column(Float)      # Raw probability (0–1)
    risk_level     = Column(String(20)) #
nominal/low/moderate/high/critical
    root_cause     = Column(String(50)) # Diagnosed failure mode
    report_md      = Column(Text)       # Full Markdown diagnostic for
operators

    # Machine-readable (for retraining & analysis)
    raw_data       = Column(JSONB)      # Original input metrics
    shap_values    = Column(JSONB)      # Top-10 SHAP contributors

    # Alert Fatigue tracking (Paper 3)
    acknowledged   = Column(Boolean)    # Was this alert seen?
    false_positive = Column(Boolean)    # Labeled post-fact
```

```
    action_taken   = Column(Text)       # What maintenance was performed?

    # Composite indexes for production query patterns
    __table_args__ = (
        Index('ix_alerts_machine_date',  machine_id, func.date(timestamp)),
        Index('ix_alerts_rootcause_fp',  root_cause, false_positive),
        Index('ix_alerts_risk_time',     risk_level, created_at.desc()),
    )
```

JSONB for `raw_data` and `shap_values`: Enables schema-free storage of variable-length diagnostic data while supporting PostgreSQL's JSON query operators (`->`, `->>`, `@>`). In production, `shap_values` can be queried to find "all alerts where tool_wear contributed >0.3 to the prediction" — a DBA-facing analytics capability.

`AlertFatigueMetrics` precomputed aggregates: The fatigue analysis query ("last 30 days by machine") on raw `alerts` at scale scans millions of rows. Precomputed daily aggregates reduce this to 30 rows → ~5ms.

3.4 API Service — Train-Serve Consistency

The most common silent failure in production ML is **train-serve mismatch**: the training pipeline computes rolling features using `groupby().rolling()` with weeks of history; the serving endpoint computes them from a single observation.

SentinelX solves this with a `FeatureStore` — an in-memory per-machine observation buffer:

```
class FeatureStore:
    """Per-machine observation buffer for computing real rolling/lag
features."""

    def __init__(self, buffer_size: int = 36):
        self.buffers: Dict[str, deque] = {}  # {machine_id:
deque(maxlen=36)}

    def update(self, machine_id: str, observation: Dict) -> None:
        if machine_id not in self.buffers:
            self.buffers[machine_id] = deque(maxlen=self.buffer_size)
        self.buffers[machine_id].append(observation)

    def compute_rolling_features(self, df: pd.DataFrame) -> Dict[str,
float]:
        """
        If >= 6 observations: real rolling statistics (matches training
exactly).
        If < 6 observations: synthetic variance based on value extremity
from
                             normal operating ranges (cold-start handling).
        """
        n_observations = len(df)
        use_synthetic = n_observations < self.min_history
```

```
        for col in rolling_cols:
            for window in [6, 12, 36]:
                mean_val = series.rolling(window=window,
min_periods=1).mean().iloc[-1]
                if use_synthetic:
                    std_val = self._estimate_anomaly_std(current_value,
col)
                else:
                    std_val = series.rolling(window=window,
min_periods=2).std().iloc[-1]
```

Cold-start problem: On a fresh machine with no history, `_estimate_anomaly_std` synthesizes variance proportional to how far the value is from normal operating ranges. Values within normal range produce low std (stable); values outside produce high std (anomalous) — preserving the model's decision boundary logic until real history accumulates after ~30 minutes.

3.5 FastAPI Inference Pipeline

```
POST /predict → engineer_features() → DiagnosticEngine.diagnose_with_llm()
                                    → create_alert() [PostgreSQL]
                                    → PredictResponse (JSON + Markdown)
```

```
@app.post("/predict", response_model=PredictResponse)
async def predict(request: PredictRequest, db: Session = Depends(get_db)):
    # Pydantic validates all fields with physics-based bounds at parse time
    # ge=200, le=400 for temperature — rejects garbage before any
computation

    features = engineer_features(request.system, request.application)

    try:
        result = await diagnostic_engine.diagnose_with_llm(
            features=features,
            machine_id=request.system.machine_id,
            timestamp=timestamp
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Diagnostic engine
error: {str(e)}")

    alert = create_alert(db=db, ...)  # Persisted for Alert Fatigue
analysis

    return PredictResponse(
        risk_level=result.risk_level,
        failure_probability=result.failure_probability,
        markdown_report=result.to_markdown()
    )
```

Connection pooling — QueuePool:

```
engine = create_engine(
    DATABASE_URL,
    poolclass=QueuePool,
    pool_size=5,          # Baseline connections
    max_overflow=10,      # Burst capacity (15 total max)
    pool_pre_ping=True,  # Detects stale connections (AWS RDS has idle
timeout)
)
```

New TCP connection per request costs ~10ms. Pool reuse costs ~0.1ms. At 100 req/s, this is the difference between a 1-second response budget being consumed by connection overhead alone.

Layer 4: DataOps & Orchestration

4.1 CI/CD Pipeline

```
# .github/workflows/ci.yml
name: SentinelX CI
```

```
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: sentinelx
          POSTGRES_DB: sentinelx
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready
          --health-interval 10s

    steps:
      - uses: actions/checkout@v3
      - name: Install Python dependencies
        run: pip install -r requirements.txt

      - name: Run unit tests
        run: pytest tests/unit/ -v --cov=src/

      - name: Run E2E tests (Playwright)
        run: npx playwright test tests/e2e/
```

CD pipeline triggers on merge to `main`:

1. Build Docker images (`Dockerfile.api`, `Dockerfile.dashboard`)
2. Push to ECR with git SHA tag (immutable artifact reference)
3. Update ECS task definition with new image digest
4. Blue/green deployment with ALB traffic shifting
5. Health check validation before 100% cutover

4.2 Observability & Monitoring

Three-tier observability model:

Tier 1 — Infrastructure (CloudWatch):

- ECS task CPU/memory utilization
- RDS connection count and query latency
- ALB 4xx/5xx error rates and response time percentiles (p50, p95, p99)

Tier 2 — Application (structured logging):

```
2024-01-15 14:23:01 [INFO]    SentinelX.API          - Processing
prediction for machine=M1
2024-01-15 14:23:01 [INFO]    SentinelX.API          - Feature store: M1
```

```
has 12 observations
2024-01-15 14:23:01 [WARNING] SentinelX.StreamManager - DRIFT DETECTED:
{'torque_Nm': 0.234}
2024-01-15 14:23:01 [ERROR]   SentinelX.Database      - Connection pool
exhausted: 15/15 used
```

Named loggers enable Splunk/CloudWatch Insights queries: `filter logger='SentinelX.StreamManager' AND level='WARNING'` isolates drift events without full-log scan.

Tier 3 — Model (business metrics via `/fatigue` endpoint):

```
{
  "total_alerts": 142,
  "alerts_per_day": 20.3,
  "false_positive_rate": 1.4,
  "paper3_benchmark_fp_rate": 54.0,
  "acknowledgment_rate": 78.5
}
```

A rising `false_positive_rate` is the primary model degradation signal — more sensitive than accuracy drift because it directly measures operator trust erosion.

4.3 Health Check Architecture

```
@app.get("/health", response_model=HealthResponse)
async def health_check():
    """Used by: AWS ALB health checks, Kubernetes liveness/readiness
probes."""
    db_health = check_db_health()
    return HealthResponse(
        status="healthy" if db_health["status"] == "healthy" else
"degraded",
        database=db_health,
        model_loaded=diagnostic_engine is not None
    )

def check_db_health() -> Dict[str, Any]:
    try:
        db = SessionLocal()
        db.execute(text("SELECT 1")).fetchone()  # Verifies live connection
        alert_count = db.query(Alert).count()
        return {
            "status": "healthy",
            "connection_pool": f"{engine.pool.size()}/{engine.pool.size() +
engine.pool.overflow()}",
            "alert_count": alert_count
        }
    except Exception as e:
```

```
        logger.error(f"Database health check failed: {e}")
        return {"status": "unhealthy", "error": str(e)}
```

ALB health check behavior: AWS ALB polls `/health` every 30 seconds. Two consecutive non-200 responses removes the task from the target group. `pool_pre_ping=True` ensures stale connection detection before the health check fails — preventing a false unhealthy state from an idle RDS connection.

4.4 Incident Response Runbook

Scenario 1: PSI drift alert

```
TRIGGER:   logger.warning("DRIFT DETECTED: {'torque_Nm': 0.234}")
SEVERITY:  P2 — model predictions unreliable within 24–48 hours

Response:
1. Check sensor calibration logs for affected machine
2. Run evaluate_and_scale.py on latest 7-day data window
3. If PR-AUC drops >5% from baseline: trigger model retrain
(train_model.py)
4. Deploy new model via CD pipeline (ECS rolling update)
5. Monitor false_positive_rate via /fatigue for 24 hours post-deploy
```

Scenario 2: Database connection pool exhaustion

```
TRIGGER:   ERROR — Connection pool exhausted: 15/15 used
SEVERITY:  P1 — API requests failing

Response:
1. Check active query count: SELECT count(*) FROM pg_stat_activity
2. Identify long-running queries: pg_stat_activity WHERE state='active' AND
duration > 5s
3. Kill blocking sessions if non-critical
4. Scale RDS instance class if sustained load
5. Increase pool_size in engine config via environment variable
```

Scenario 3: Model serving cold-start

```
TRIGGER:   Feature store shows machine with 0 observations (new deployment)
BEHAVIOR:  Synthetic variance used for cold-start
(FeatureStore._estimate_anomaly_std)
RESOLUTION:After 6 observations (~30 minutes), real rolling stats activated
automatically
MONITORING:Log "Feature store: {machine_id} has {n} observations" — alert
if stuck at 0
```

4.5 AWS Production Migration Checklist

The system is designed as a **Digital Twin** — local Docker and AWS ECS are structurally identical, differing only in environment variables:

Local	AWS	Config mechanism
`localhost:5432` (Docker PostgreSQL)	RDS endpoint	`DB_HOST` env var
`localhost:11434` (Ollama)	`host.docker.internal:11434`	`OLLAMA_URL` env var
`.env` file	AWS Secrets Manager	`dotenv` / IAM role
`docker-compose up`	ECS Service + ALB	Task definition

Migration reduces to: push images to ECR, create RDS instance, store secrets, create ECS services. Zero code changes.

Appendix: Pipeline Stage Reference

Stage	Module	Input	Output	Key Engineering Decision
1	`stream_manager.py`	Raw CSV	Quality-flagged chunks	Generator pattern, FLAG not DROP
2	`feature_engineer.py`	Quality chunks	Feature matrix (105 cols)	Per-machine lookback buffer
3	`train_model.py`	Feature matrix	`model.joblib` + SHAP	SMOTE inside CV loop only
4	`autoencoder.py`	Feature matrix	Anomaly scores	Baseline comparison only
5	`evaluate_and_scale.py`	Model artifacts	Threshold config	PR-AUC primary metric, not accuracy
6	`agent.py`	Prediction + SHAP	Markdown diagnostic	LLM with rule-based fallback
7	`main.py` + `database.py`	HTTP request	Alert record + JSON	FeatureStore fixes train-serve mismatch
8	`dashboard.py`	PostgreSQL	Streamlit UI	Precomputed fatigue aggregates

---

SentinelX Technical Design Document — v1.0